

MettaKernel: A MeTTa-hosted Proof/Inference Kernel for both Dependent Type Theory and Higher-Order Tarski–Grothendieck Set Theory

Design, Curriculum, and a Prototype-First Build Plan

Oruži

Zar Goertzel

2026-06-18 (design draft); realized-architecture update 2026-06-23;
Metamath-profile update 2026-08-06

Abstract

We set out to build one small *trusted* proof/inference kernel, written *in MeTTa* (as an ordinary MeTTa program, executed on the CeTTa engine), that can express and check proofs in *both* foundations we care about: CIC-grade *dependent type theory* (the foundation of Lean and Coq) and *higher-order Tarski–Grothendieck set theory* (HOTG, the classical higher-order logic of Megalodon). The kernel is a MeTTa program — not a new CeTTa language substrate — in the same spirit as `pverify`, an existing Metamath proof checker written in MeTTa. Two principles govern the work. First, **prototype a fully working kernel before any deeper integration into the engine**, so that the design is chosen empirically rather than frozen in advance. Second, **conversion is the danger line**: the kernel’s $\beta\eta\delta\iota$ conversion must be its own small, decidable, resource-bounded MeTTa functions and must never be delegated to CeTTa’s general runtime rewriting, or proof checking ceases to be a terminating judgment. We present the two reference kernels we mirror (the Lean `PureKernel` for DTT and Megalodon’s `tp/tm/pf+check_doc` core for HOTG), a graded *curriculum* of the easiest examples that pin down the smallest fully expressive kernel (a DTT ladder following Smolka & Brown’s *Introduction to Computational Logic*, and an HOTG ladder following Megalodon), and a prototype-first build plan whose first milestone is an honest *capacity ledger* (AGREE/DIVERGE/OPEN) rather than a claim of finality. The MeTTa kernel’s verdicts are treated as a *subject* to be measured against the oracles, not as an oracle itself.

1 Why a single kernel for two foundations

Lean/Coq rest on the Calculus of Inductive Constructions: dependent function and pair types, an inductive-definition mechanism with computation rules, a universe hierarchy, and a *decidable* definitional equality. Megalodon rests on HOTG: simple types with an impredicative `Prop`, proof terms, classical logic, set membership, extensionality, foundational ($\in$ -)induction, and a choice operator. These are different mathematics, but a *proof kernel* for either is small: it is a term/type/proof object language plus a conversion relation plus a checking judgment, with everything else (parsing, automation, optimization, the ambient runtime) living *outside* the trusted core.

Our thesis — developed at length in the companion memo *metta_proof_kernel_grounded* — is that MeTTa is an apt host for such a kernel *as a program*: terms, types, and proofs are MeTTa atoms; the checking and conversion judgments are MeTTa functions; theories and imports are MeTTa data. The trusted waist is then a small auditable MeTTa module, and the two foundations appear as two *checkers* sharing that waist. We are not proposing to encode both foundations into

one ambient rewriting theory (the Dedukti route); we keep the foundations’ judgments first-class and only share the machinery beneath them.

What this document is. A design + curriculum memo to drive a prototype-first build. It records: the reference kernels we mirror (§2); the MeTTa-program architecture (§3); the graded curriculum that defines “smallest fully expressive” (§4); the build plan and its first milestone, a capacity ledger (§5); the conversion discipline that is the central hazard (§7); the open questions we deliberately leave open (§8); and — added in this revision — the *realized architecture and results* (§6) recording the as-built kernel that now exists, including where realization refined the plan.

2 The two reference kernels (our oracles and templates)

2.1 DTT: the Lean PureKernel

A trusted, *sorry-free* dependently-typed kernel already exists in Lean (Mettapedia/.../MeTTa/PureKernel/, 28 files). Its term language is

```
PureTm n ::= var (Fin n) | const DeclName | U0 | U1
          | pi A B | sigma A B | id A t u
          | lam B | app f a | pair t u | fst p | snd p | refl t
```

with two universes (U0 : U1), Π , Σ , and identity types. It provides one-step $\beta/\delta/\iota$ reduction, Church–Rosser confluence, HasType, a *decidable* definitional equality (defEqByNormalization?), subject reduction, and a bidirectional checker. Ordinary families (Bool, Nat, Unit) are *not* hard-coded: they enter through a general declaration mechanism (DeclSpec, InductiveDecl, RecursorDecl) with full well-formedness proofs. This is both our DTT *oracle* and our design template; crucially, the existing runtime-to-kernel bridge marks directExecEquivalence as false, so the runtime is *not* yet known to agree with this kernel.

2.2 HOTG: the Megalodon mathdata core

Megalodon’s trusted core (repos/megalodon-1.13/src/mathdata.mli) is genuinely compact — three small inductive types and a checker:

```
tp ::= TpVar i | Prop | Base i | Ar(tp, tp) | TpAll tp          (* simple types + type-vars
+ type-forall *)
tm ::= DB i | TmH h | Prim i | Ap(tm, tm) | Lam(tp, tm)
      | Imp(tm, tm) | All(tp, tm) | TTPAp(tm, tp) | TTPLam tm | TTPAll tm
pf ::= Gpa h | Hyp i | Known h | PTmAp(pf, tm) | PPfAp(pf, pf)
      | PLam(tm, pf) | TLam(tp, pf) | PTPAp(pf, tp) | PTPLam pf
```

plus check_doc (the proof checker), $\beta\eta\delta$ normalization with resource limits (BetaLimit, NonNormalTerm), and hash-addressed doc/theory/signa imports. Logic and set-theoretic content are *not* in this core: True/False/not/and/or are impredicative definitions, and membership/extensionality/choice/ $\in$ -induction enter as *axioms* in a preamble (Egal mode uses Eps choice and Tarski’s axiom; Mizar mode uses the and ZF-style axioms). The seed file Mettapedia/megalodon/experiments/01_basics.mg proves eight propositional theorems entirely by proof-term lambdas (e.g. andEL := fun A B H => H A (fun a b => a)).

2.3 Precedent that MeTTa can host a checker

hyperon/metamath/pverify (with `lib/pverify-utils.metta`) is a Metamath proof checker realized as a MeTTa program. It establishes feasibility of the *host*; it is Metamath, not HOTG, so it is precedent, not evidence about the present design.

3 Architecture: the kernel as a MeTTa program

Shared waist (written once in MeTTa). Atom encodings for binders (de Bruijn or locally nameless), α -safe substitution, typed contexts, declarations (parameter / definition / axiom / theorem / inductive / import), proof objects, the **conversion engine** ($\beta/\eta/\delta/\iota$ as explicit, terminating, resource-capped MeTTa functions), a **check/infer** driver, and hashing/provenance for imports. The two checkers below are *instances* of this waist — that sharing *is* the early unification.

HOTG checker. Mirror `tp/tm/pf + check_doc +  $\beta\eta\delta$` as MeTTa atoms and functions. Proof-term checking over impredicative **Prop**; classical/set/TG axioms are a *profile* layered above the core, admitted only after the bare proof-term checker is green.

DTT checker. Mirror **PureTm** (`U0/U1,  $\Pi, \Sigma, Id$`) with bidirectional `infer/check`, $\beta/\delta/\iota$, and decidable definitional equality by normalization; inductive families via **DeclSpec**, never hard-coded. Where the CeTTa engine’s `-profile he-prime` already provides dependent type inference, the MeTTa kernel may use it; new C features are added to **he-prime** *only* when the prototype proves a concrete need.

4 Curriculum: the examples that pin the smallest fully expressive kernel

The curriculum *defines* “fully expressive”: the smallest kernel is the one that checks every example below. The DTT ladder follows the chapter order of Smolka & Brown’s *Introduction to Computational Logic* (a Coq course by Megalodon’s own author), which introduces kernel features in increasing dependency order. The HOTG ladder follows Megalodon.

L0 — shared waist (both ladders ride on it). Names/binding, α -safe substitution, typed contexts, declarations, proof objects, conversion ($\beta/\eta/\delta/\iota$, decidable), hash imports + provenance, negative tests.

Table 1: DTT ladder (ICL spine + CeTTa **he-prime** seeds + **PureKernel**).

Step	Example (source)	Kernel feature it pins
D1	<code>bool/negb/andb/andb_com</code> (ICL §1.1)	inductive Bool , <code>match/recursor</code> , <code>def-eq</code> by computation
D2	<code>plus_0/plus_S/plus_assoc</code> (ICL §1.3–1.4)	Nat , <code>recursor</code> , structural induction
D3	<code>Π / proofs-as-programs</code> (ICL §1.2, 2.2, 2.6)	Π formation/intro/elim, application

Step	Example	Kernel feature
D4	Σ / pairs (ICL §6.6; he-prime TutPackHuman)	Σ formation/intro/projection
D5	Id/refl/cong/transport (ICL §3.3; he- prime TutEq/TutRefl/TutSuccCong)	identity types, J /transport
D6	app_assoc/length_app (ICL §8)	parametric inductive + recursor
D7	Vec/TutIndex safety; VecN -1 (open)	indexed inductive families; index-domain question (Number vs Nat)
D8	Even/ $\leq$ (ICL §7)	inductive predicates, elim restriction
D9	recursors + elim restriction + universes (ICL §5)	ι -reduction, proof irrelevance, universe consis- tency

Table 2: HOTG ladder (01_basics.mg + Megalodon +
mathdata tp/tm/pf).

Step	Example (source)	Kernel feature it pins
H1	propositional encodings (01_basics.mg, 8 thms)	tp(Prop), tm(All, Imp, Lam, Ap), pf (PLam, PPfAp, Hyp), β -normalization, check_doc
H2	$\forall$ -instantiation; $\rightarrow$ /proof application	PTmAp, PPfAp
H3	proof λ ; $\beta\eta\delta$; definition unfolding	δ on ThyDef/SignaDef, η
H4	equality/congruence (Leibniz eq)	polymorphic eq via TpAll
H5	membership/set/TG axioms (In/Empty/EmptyE, set-ext, In_ind, Eps)	<i>axiom profile</i> , admitted only after H1–H4 green

Cross-profile smoke. $P \rightarrow P$; $P \rightarrow Q \rightarrow P$; $P \wedge Q \rightarrow P$; $\perp \rightarrow P$; equality reflexivity/congruence; pair projections; and *one* tiny CeTTa operational-semantics theorem treated as a *certificate*, never as definitional equality.

Negative corpus (a fail-by-default gate — the soundness backstop). Bad de Bruijn index; unknown Known/Hyp; application type mismatch; a proposition-shape used as a proof object; cyclic/non-normal δ ; **an arbitrary MeTTa rewrite admitted as conversion** (the central danger); stale import/hash; VecN ...-1 wrongly accepted. The kernel must *reject* every item here; a checker that accepts any of them is unsound.

5 Prototype-first build plan

Phase 0. Shared MeTTa waist + the ledger harness (three deciders: MeTTa-kernel vs. Lean PureKernel oracle vs. he-prime C).

Phase 1 (HOTG prototype). MeTTa tp/tm/pf + proof-term checker + β ; *check the eight 01_basics.mg theorems green* (H1–H2).

Phase 2 (DTT prototype). MeTTa PureTm + bidirectional checker + $\beta/\delta/\iota$ + decidable def-eq; *D1–D5 green*; ledger vs. the Lean oracle and he-prime C.

Phase 3 (unify + harden). Both checkers share the waist; cross-profile smoke green; *negative corpus rejected*.

Phase 4 (climb the ladders). HOTG H3–H5 (definitions, equality, then set/TG axioms); DTT D6–D9 (lists, `Vec`, inductive predicates, recursors/universes); resolve the `VecN` index-domain question.

Later. Only after a fully working prototype and its lessons: consider deeper substrate integration — add the `he-prime` C features the prototype proved necessary; refine CeTTa. Not before.

Milestone 1 (the ledger, not a constitution). The deliverable is the working H1 + D1–D3 MeTTa kernel plus a populated *capacity ledger* with buckets AGREE-ACCEPT, AGREE-REJECT, DIVERGE, OPEN, plus the negative-corpus rejections. Divergences and open cases are first-class outputs, not blemishes to hide. Only then does external review become useful, and its question is whether the fragment boundary is honest and which divergences are bugs versus legitimate language semantics — not whether the final kernel is settled. The MeTTa-kernel verdicts are labelled **subject**; the Lean `PureKernel` and Megalodon are the **oracles**.

6 Realized architecture and results (June 2026)

The prototype is built and runs. This section records the *as-built* kernel and, honestly, where realization *refined* the plan above; the earlier sections stand as the original intent.

One $\lambda\Pi$ -modulo checker; object logics as signatures. Rather than two separate term languages (`PureTm` and `tp/tm/pf`), the realized kernel is a *single* $\lambda\Pi$ -style logical-framework checker (`Srt/Con/Var/Pi/Lam/App/Def`) with one `infer/check/conv`; each object logic enters as a *signature* of typed declarations over that one checker. DTT, HOTG, and HOL all appear in `kernel/curriculum_lf_demo.metta` as signatures over the same checker, each with positive proofs and a rejected negative. This keeps every logic’s rules first-class (as signature declarations) while sharing the trusted waist — LF-hosting, deliberately distinct from desugaring every logic into one ambient rewrite theory.

One generic inductive engine (DIndG). The hard-coded inductive families of the DTT ladder (D6–D9) are subsumed by a single generic *indexed-inductive* declaration, `DIndG`: a telescope IR (parameters Δ , indices Ξ , constructors) from which the family kind, constructor types, the eliminator, branch types, and constructor-headed ι -rules are *generated and checked*. `Nat`, `Bool`, `List`, `Tree`, `Pair`, `Vec`, `Fin`, and `Id` are all instances; crucially `Id` is itself a `DIndG` and `J` is its *generated* (diagonal) recursor, so the equality primitives and six special-case inductive slices retired onto the one engine. (This resolves the original `VecN`/index-domain open question: indices are an arbitrary telescope, not a single `Nat`.)

Binding as a declared property (the ABT waist). Binding lives in one import-free waist (`kernel_binding_waist_v1`): de Bruijn under the hood, with capture-avoiding `shift/subst` written once and *arity-parameterized*; clients delegate to it rather than re-implementing substitution. A `BindingDecl` layer (`kernel_binding_decl_v1`) lets each hosted `LanguageDef` *declare* its binders (`Bind0/Bind1/BindFields/BindPi`: dependent Π , $\forall/\exists$ bodies, comprehension predicates, proof binders) without disturbing the hot waist. Capture-avoidance is mutation-gated.

Admission, checked-once theorems, and the admit-once lesson. A signature is *admitted* only if every declaration is well-formed: constants’ types check, definitions’ bodies check against their declared types (`wf-decl`), and inductive declarations are fully generatable. Two declaration forms serve derived libraries: `DAxiom` (an honest postulate, recorded in *axiom provenance*) and `DThm` (a *checked* theorem constant — its proof body is checked *once* at admission, then used opaquely by its type, transparent only for provenance). A memory hazard forced the central engineering lesson: naively, `kernel-check` re-ran full signature admission on *every* theorem check, so a file checking 21 theorems against a signature carrying two large derived-proof bodies re-normalized those bodies 21 times and exhausted memory (~ 48 GiB, OOM). The fix is *admit-once batch checking* — admission computed once, lexically, for a batch of jobs, with *no* forgeable “checked-signature” token exposed (in MeTTa a token is plain data anyone could fabricate). This is exactly HOL-Light’s discipline (a derived rule is a program calling primitives, never one giant proof term re-normalized from scratch) and it dropped the same file to ~ 8 GiB.

Faithful object-logic hosting: HOL-Light’s equality kernel. HOL is hosted not as a convenient natural-deduction surrogate but on HOL-Light’s *actual* primitive basis — `REFL`, `TRANS`, `MK_COMB`, `ABS`, `BETA`, `ASSUME`, `EQ_MP`, `DEDUCT_ANTISYM` as LF rule certificates checked by the imported kernel — with $\top/\wedge/\Rightarrow$ defined over equality and λ (matching `bool.ml`) and derived `MP/DISCH` shards admitted as checked `DThms`. Each primitive and each shard carries a mutation tooth. *Faithfulness caveat:* the definitional equations are currently *postulated* as constants matching `bool.ml` rather than introduced through a checked conservative-definition principle, and the derived `MP/DISCH` are specific checked replays, not yet a fully general derived-rule library.

The curriculum as a calibration matrix. The curriculum is now an explicit ledger (`Curriculum/oracle_cases.tsv`): each row is a kernel capacity $\times$ profile (DTT/HOL/HOTG/Metamath/...) $\times$ the SOTA system that calibrates the verdict (Lean/Coq/Agda/Megalodon/HOL-Light), with status `CHECKED` or `PLANNED` and a per-row gate. The external systems are the *calibration oracle*; the destination is to *host each core logic as a LanguageDef and re-prove the curriculum’s theorems in our own kernel*. Verdicts are differential — a row borrows its expectation from the source system, not from our own guess.

The Metamath profile (August 2026 update). The Metamath row of the matrix is now the furthest along the stated destination. The full statement language is authored twice at the `LanguageDef` level — once in `languageDef!` form (`metamathCore`, including the front-end lowering pipeline as rewrite rules) and once as the proof-facing `sourceGrammar`, whose presentation passes the generic validator and whose accepted pipeline runs derive the outer database sort in the lexicalized language of their own classified streams with no supplied lexical premises (`runSource_selfLexicalized_derives`). On the semantic side, every accepted source declaration transition is matched by the corresponding shipped `mm-lean4` database operation, and the whole statement fold co-evolves with the implementation database from the initial states (`default_initial_applyPayloads`), including the identification of the implementation’s mandatory-frame trim with the specification’s mandatory frame (`trimFrame’_eq_mandatory`). Checked source admission runs exact bytes through the actual verified reader into one generated presentation package. Deliberately open, in the obligations ledger’s sense: per- $\$p$ proof discharge from the reader’s own chronology (normal lane recently constructed; compressed in progress), the distinction between verified and ?-incomplete outcomes as disjoint results, and the prefix-wise source/reader simulation in which reader microsteps are administrative. The Metamath lane

thereby serves as the template for how the other profile rows are intended to mature.

Validation discipline. Every checkpoint runs on three engines — CeTTa (C), PeTTa (Prolog), and the Lean-verified *LeaTTa* as a deterministic oracle — in parity, with per-feature mutation gates (corrupt the rule, require red) and the fail-by-default negative corpus, all behind one `run_all` gate; heavy runs are memory-capped.

Soundness status (honest). Admission plus provenance close the per-input declaration holes (a `break_probes` probe documents what the bare term-checker would accept *without* admission; the real kernel rejects those). The *one* genuinely-open soundness obligation is metatheoretic, not a per-input check: **universal subject-reduction, confluence, and termination of the generated ι -rules are not yet machine-proven** — we have an executable SR-wedge self-test and the LeaTTa engine-floor, not the theorem. That metatheorem is the live Lean target. The kernel is today *checked-correct per input* but not yet *proven-correct as a calculus*.

7 Conversion discipline (the central hazard)

The single biggest architectural risk is identifying kernel conversion with general MeTTa evaluation. We hold five lines: (A) decidable conversion $\neq$ general MeTTa eval — the kernel’s $\beta/\eta/\delta/\iota$ are its own bounded MeTTa functions (resource caps in the spirit of Megalodon’s `BetaLimit`); (B) MeTTa’s many-valued/nondeterministic typing vs. a functional DTT judgment — the kernel commits to a functional infer/check; (C) grounded/FFI values are a trusted boundary — grounded equality is not kernel reduction; (D) universe consistency is not optional forever, because proofs *about* MeTTa are a goal; (E) the kernel must be small and separable — a pure checking slice, not entangled in effectful evaluation.

8 Open questions (kept deliberately open)

Resolved or refined by the realized architecture (§6): the `VecN/index-domain` question (indices are an arbitrary telescope under `DIndG`); the first dependent-typing fragment ($\lambda\Pi + \text{DIndG}$ indexed inductives, predicative); the trust boundary (it is signature *admission*); and the convergence target (the kernel is its own $\lambda\Pi\text{-LF}$, with `PureKernel`/Megalodon as templates and the Lean-verified LeaTTa as the runnable oracle floor, rather than `PureKernel` itself).

Live and deliberately open: (1) the **universal subject-reduction / confluence / termination metatheorem** for the generated ι -rules — the single open soundness obligation, now a Lean target (the toolchain is unblocked at v4.31); (2) **engine-level term sharing** (hash-consing / DAG / normalization-by-evaluation in CeTTa) for deeper scaling, of which admit-once batch checking is the cheaper architectural half already in hand; (3) a **checked conservative-definition principle** so hosted definitional equations are derived rather than postulated; (4) a **fully general derived-rule library** (beyond specific checked replays); and (5) **breadth** — hosting further core logics as LanguageDefs (fuller HOTG via Megalodon, then Lean-core/CIC) over the now-shared, now-affordable waist.

References (local anchors)

- G. Smolka and C. E. Brown, *Introduction to Computational Logic*, Lecture Notes, Saarland University, 2014. (DTT curriculum spine.)

- *metta_proof_kernel_grounded* (AIXiv, 2026): framework core, object-theory profiles, conversion discipline, hash imports.
- Megalodon: `repos/megalodon-1.13/src/mathdata.mli` (the HOTG `tp/tm/pf` + `check_doc` core); `examples/` (Egal/Mizar preambles); `experiments/01_basics.mg` (seed).
- Lean `PureKernel/README.md` and modules (the DTT oracle/template).
- `hyperon/metamath/pverify` (MeTTa-hosted checker precedent).